Java Basics — Section 1: Foundations

Oct 27, 2026 · @Kanhaiya

1.1 Introduction to Java

Java is a high-level, class-based, object-oriented programming language built around one core promise: **Write Once, Run Anywhere (WORA)** — compile your code once and run it on any machine that has a JVM, without changing a line.

What is Java?

Java is both a **language** and a **platform**. As a language, it defines syntax, keywords, and rules for writing programs. As a platform, it provides the runtime environment (the JVM) that actually executes those programs. This distinction matters — other languages like Kotlin and Scala also compile to Java bytecode and run on the Java platform.

Java is **statically typed** (types are checked at compile time), **strongly typed** (no implicit conversions that could cause data loss), and **case-sensitive** (`main` and `Main` are different identifiers).

Why Java?

- **Platform independence.** Java compiles to bytecode, not native machine code. Any machine with a JVM can run the same `.class` file — Windows, Linux, macOS.
- **Object-Oriented.** Encourages organized, reusable, and maintainable code through encapsulation, inheritance, polymorphism, and abstraction.
- **Robust.** Strong compile-time type checking, exception handling, and automatic memory management (garbage collection) prevent large categories of bugs.
- **Secure.** The JVM sandbox, bytecode verifier, and module system prevent unauthorized memory access.
- **Huge ecosystem.** Spring, Hibernate, Maven, Gradle, Kafka, Spark — Java has frameworks for nearly every domain.
- **Enterprise standard.** Banking, e-commerce, government, Android apps, and big-data pipelines run heavily on Java.

When to Use Java?

- Backend REST APIs and microservices (Spring Boot, Quarkus)
- Enterprise applications requiring high reliability and scalability
- Android mobile development
- Big data processing (Hadoop, Spark, Kafka)
- Financial systems where auditability and correctness matter most

History of Java — Key Milestones

Year	Milestone
1991	James Gosling at Sun Microsystems starts Project Green, targets consumer electronics. Language named **Oak**.
1995	Renamed **Java**. Released publicly. Browser applets create excitement about interactive web.
1998	**Java 2 (1.2)** — Collections Framework, Swing, JIT compiler. Three editions introduced: J2SE, J2EE, J2ME.
2004	**Java 5** — Generics, enhanced for-each, autoboxing, enums, annotations, varargs. Major language update.
2006	Sun open-sources Java under GPL. **OpenJDK** created as the official open-source reference implementation.
2011	**Java 7** — try-with-resources, diamond operator `<>`, binary literals, underscore in numeric literals.
2014	**Java 8 (LTS)** — Lambdas, Stream API, Optional, new Date/Time API, default interface methods. The biggest release since Java 5.
2017	**Java 9** — Module system (Project Jigsaw), JShell REPL, collection factory methods. New 6-month release cadence begins.

Year	Milestone
2018	**Java 11 (LTS)** — `var` in lambdas, new String methods, standardized HttpClient.
2021	**Java 17 (LTS)** — Sealed classes, records, pattern matching for instanceof, text blocks become standard.
2023	**Java 21 (LTS)** — Virtual threads, pattern matching for switch, record patterns, sequenced collections.

Release model (Java 9+): A new version every 6 months (March and September). Most versions are short-lived (supported 6 months). An **LTS (Long-Term Support)** version is released every 2 years and supported for 8+ years. Always use LTS in production: Java 8, 11, 17, 21, and Java 25 (released September 2025).

Java Editions

Java SE (Standard Edition) is the core platform — the language, JVM, and standard libraries (`java.lang` , `java.util` , `java.io` , `java.net` , etc.). Everything else builds on top of it.

Jakarta EE (formerly Java EE) adds enterprise APIs on top of SE — Servlets, JPA (database), JAX-RS (REST), CDI (dependency injection), JMS (messaging). In 2017, Oracle transferred Java EE to the Eclipse Foundation, which renamed it Jakarta EE. In practice, most teams use **Spring Boot** instead of raw Jakarta EE.

Java ME (Micro Edition) is a stripped-down Java SE for resource-constrained embedded and IoT devices. Largely replaced by Android (which uses a modified JVM called ART).

Features of Java

1. Platform Independence — The WORA Promise

A C program compiled on Windows produces a Windows `.exe` — it will not run on Linux. Java is different:

```
.java source file
      ↓  javac (compiler)
.class file (bytecode — platform-independent)
      ↓  JVM (platform-specific interpreter/JIT)
 Runs on any OS that has a JVM
```

The bytecode is NOT CPU instructions. It is an intermediate set of instructions designed for the JVM. Every OS has its own JVM implementation that translates the same bytecode to that OS's native code. Your program is written once; the JVM handles the rest.

2. Object-Oriented

Everything in Java (except primitive types) is an object. Java enforces the four OOP pillars:

- **Encapsulation** — hiding internal data behind access modifiers (`private`, `public`)
- **Inheritance** — reusing behavior from parent classes (`extends`)
- **Polymorphism** — one method name behaves differently depending on the object (`@Override`)
- **Abstraction** — exposing only essential details via `abstract` classes and `interface`

Java is NOT purely object-oriented because it has 8 primitive types (`int`, `boolean`, etc.) that are not objects, and it allows `static` methods that belong to a class rather than an instance.

3. Robust

Java was designed to minimize programmer mistakes. Strong compile-time type checking catches type errors before the program runs. The exception handling system (`try-catch-finally`) ensures programs recover gracefully from runtime errors. Garbage collection automatically reclaims memory — no manual `malloc`/`free` like in C. Array bounds checking at runtime prevents buffer overflows.

4. Secure

The JVM creates a sandbox. The **bytecode verifier** checks every `.class` file before execution to ensure it doesn't violate access rules. Java has no pointer arithmetic — you cannot read or write to arbitrary memory addresses. The module system (Java 9+) enforces strong encapsulation — internal APIs are hidden by default.

5. Multithreaded

Java has first-class support for concurrent programming through `java.lang.Thread`, the `java.util.concurrent` package, and Java 21's virtual threads (Project Loom). The **Java Memory Model (JMM)** formally defines how threads interact through shared memory.

6. Architecture-Neutral (Portable)

In C, the size of an `int` depends on the platform. In Java, every primitive type has a fixed, platform-independent size. An `int` is always 32 bits, a `long` is always 64 bits, a `char` is always 16 bits — everywhere.

7. High Performance

Early Java was slow because bytecode was interpreted. The **JIT (Just-In-Time) compiler** detects frequently-executed ("hot") code and compiles it directly to native machine code at runtime. Modern JVMs with HotSpot optimizations (method inlining, escape analysis, tiered compilation) make Java performance comparable to C++ in many server workloads.

8. Distributed

Java was designed with networking built in. The `java.net` package provides TCP/IP and UDP socket APIs. Technologies like REST, gRPC, and messaging queues make Java a natural fit for distributed systems.

9. Dynamic

Java can load classes at runtime (dynamic class loading), inspect and invoke methods via reflection, and adapt to changing environments. This is the technical foundation for frameworks like Spring (reflection-based dependency injection) and tools like serialization, proxies, and annotation processing.

Main Components of Java

Component	What it is
Java Language	Syntax, keywords, data types, control flow, OOP constructs
JDK	Complete developer toolkit — compiler, debugger, tools, + JRE
JRE	Runtime environment — JVM + standard libraries
JVM	The engine that executes bytecode

Component	What it is
Bytecode	Platform-independent intermediate code in `.class` files
ClassLoader	Loads `.class` files into JVM memory on demand
Java API	Standard library: java.lang, java.util, java.io, java.net, java.sql…

Java Program Structure

Every Java program follows this layout. Order matters — `package` first, then `import`, then the class.

```
// 1. Package declaration — optional, must be the FIRST non-comment line
package com.example.myapp;

// 2. Import statements — after package, before class
import java.util.List;
import java.util.ArrayList;
import static java.lang.Math.PI; // static import for direct name access

// 3. Class declaration
// The filename MUST match the public class name exactly (case-sensitive)
// HelloWorld.java → public class HelloWorld
public class HelloWorld {

    // 4. Fields (instance variables)
    private String message;

    // 5. Constructor
    public HelloWorld(String message) {
        this.message = message;
    }

    // 6. main method — the entry point
    // Signature must be exactly: public static void main(String[] args)
    public static void main(String[] args) {
```

```
        HelloWorld obj = new HelloWorld("Hello, World!");
        System.out.println(obj.message);
    }
}
```

Key rules:

- There can be only **one `public` class per `.java` file**, and its name must match the filename.
- `main` must be exactly `public static void main(String[] args)` to serve as the entry point.
- Java is **case-sensitive**: `Main`, `main`, and `MAIN` are three different identifiers.
- `package` before `import`; `import` before the class declaration.

Comments in Java

```
// Single-line comment — from // to end of line

/* Multi-line comment
   spans multiple lines
   ignored by compiler */

/**
 * Javadoc comment — used to generate HTML documentation.
 * Starts with /** and uses @ tags.
 *
 * @param radius  the radius of the circle
 * @return        the area as a double
 * @throws IllegalArgumentException if radius is negative
 * @author        James Gosling
 * @since         1.0
 */
public double calculateArea(double radius) {
    return Math.PI * radius * radius;
}
```

Running `javadoc HelloWorld.java` on source files generates the entire HTML API documentation — the same way Oracle's official Java documentation is produced.

Java Program Compilation and Execution

Compilation step — converts human-readable source to platform-independent bytecode:

```
HelloWorld.java
      |
      |  javac HelloWorld.java
      |
      v
HelloWorld.class   ← bytecode (NOT native machine code)
```

`javac` performs: lexical analysis → parsing → type checking → bytecode generation. If any step fails (syntax error, type mismatch), compilation stops with an error and no `.class` file is produced.

Execution step — the JVM loads and runs the bytecode:

```
HelloWorld.class
      |
      |  java HelloWorld
      |
      v
  [ClassLoader]        → loads HelloWorld.class and its dependencies
      |
      v
  [Bytecode Verifier]  → security check: is this bytecode valid and safe?
      |
      v
  [Interpreter / JIT]  → interprets bytecode OR compiles hot paths to native
code
      |
      v
  Program Output on your OS
```

This two-step approach is what makes Java platform-independent. The `.class` bytecode file is identical on every platform. Only the JVM differs per OS — it handles the final translation to native CPU instructions.

1.2 JDK, JRE and JVM — In Depth

The three components are nested: JDK contains JRE, and JRE contains JVM. Knowing this boundary tells you exactly what to install and why.

Component	Full Name	Contains	Who needs it
JVM	Java Virtual Machine	ClassLoader · Memory · Execution Engine	Embedded in JRE
JRE	Java Runtime Environment	JVM + Java standard libraries	Anyone running Java apps
JDK	Java Development Kit	JRE + compiler + all dev tools	Developers writing Java

JDK — Java Development Kit

Install the JDK to **write and compile** Java code. Key tools it includes:

- `javac` — Java compiler. Converts `.java` source files to `.class` bytecode.
- `java` — JVM launcher. Starts the JVM and runs a compiled program.
- `javadoc` — Generates HTML API docs from Javadoc comments.
- `jar` — Packages `.class` files into distributable `.jar` archives.
- `jdb` — Command-line Java debugger for stepping through running code.
- `javap` — Class file disassembler. Lets you inspect bytecode inside a `.class` file.
- `jshell` — Interactive REPL (Java 9+). Try Java expressions without a full program.
- `jlink` — Module linker (Java 9+). Creates a minimal custom JRE with only the modules your app uses.
- `jmap`, `jstack`, `jconsole` — Diagnostic tools for heap dumps, thread dumps, and JVM monitoring.

Since Java 11: Oracle no longer ships a standalone JRE. The JDK is the standard download. Use `jlink` to bundle a trimmed runtime with your app.

JVM — Java Virtual Machine

The JVM is a **specification**, not a single product. Multiple vendors implement it:

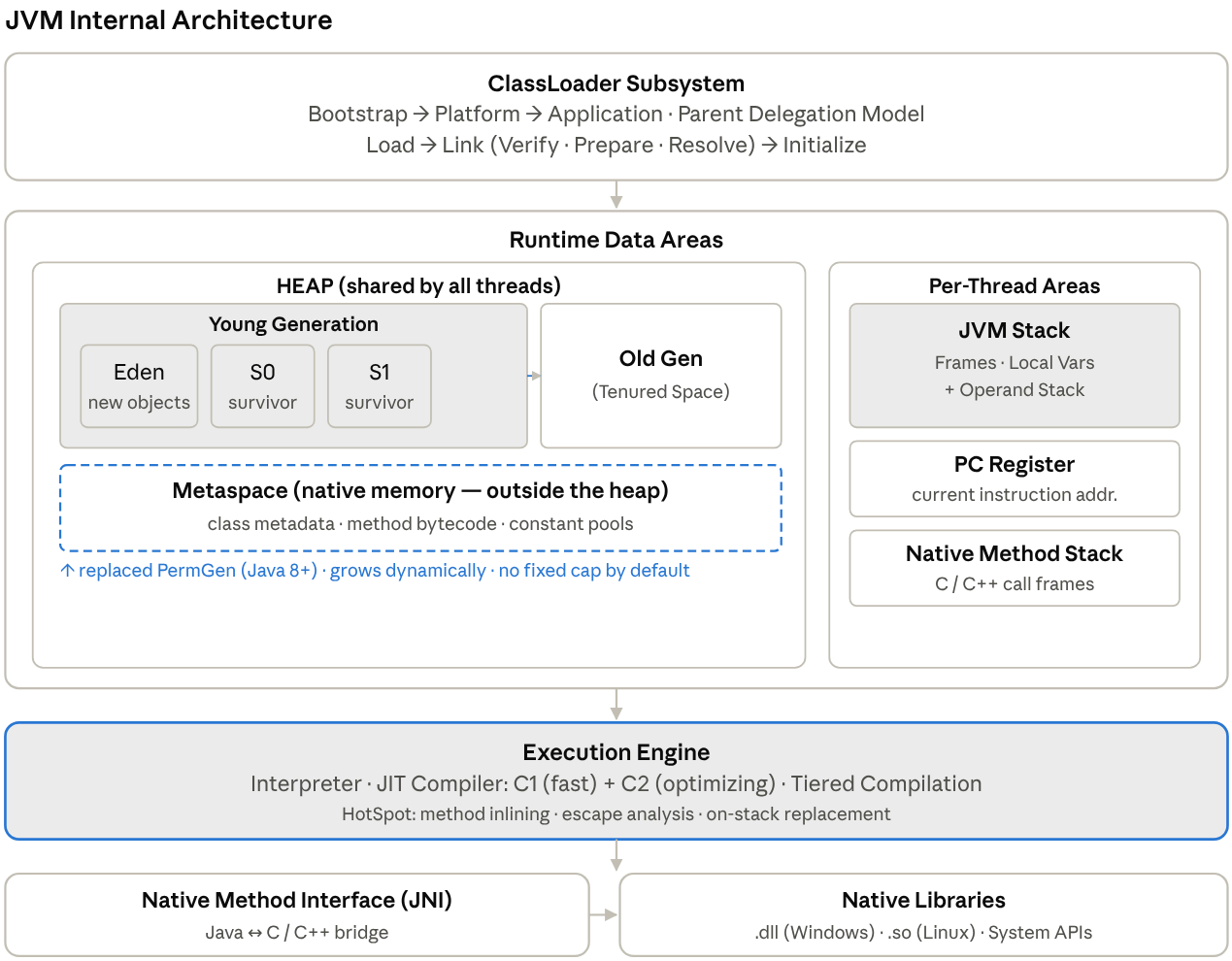
JVM	Vendor	Key strength
HotSpot	Oracle / OpenJDK	Most widely used · advanced JIT optimizations
OpenJ9	IBM / Eclipse	Low memory footprint · great for containers

JVM	Vendor	Key strength
GraalVM	Oracle	Polyglot · ahead-of-time native image compilation
Azul Zing	Azul	Ultra-low latency garbage collection

The JVM itself is **platform-specific** (the Linux JVM differs from Windows), but every JVM runs the **same** platform-independent bytecode. That is how WORA works.

JVM Internal Architecture

Every `.class` file flows through the ClassLoader into memory, gets executed by the Execution Engine, and native calls go through JNI.



JVM Architecture · ClassLoader → Memory → Execution Engine

1. ClassLoader Subsystem

The ClassLoader loads `.class` files into JVM memory before they can run. It works in three phases.

Parent Delegation Model

When a ClassLoader is asked to load a class, it **delegates to its parent first**. Only if the parent cannot find the class does the child attempt to load it.

```
Request: load com.example.MyClass
  AppClassLoader → delegates to ExtClassLoader
    ExtClassLoader → delegates to Bootstrap
      Bootstrap: not in core JDK
    ExtClassLoader: not in platform modules
  AppClassLoader: found on classpath ✓
```

The three built-in ClassLoaders:

- **Bootstrap ClassLoader** — Written in native C++. Loads core JDK classes (`java.lang.String`, `java.lang.Object`) from `java.base`. Root of the hierarchy — has no parent.
- **Platform ClassLoader** (Extension ClassLoader in Java 8) — Loads Java SE platform module classes.
- **Application ClassLoader** — Loads your application code and third-party JARs from the classpath.

Why it matters: Bootstrap always loads `java.lang.String` first. A class named `java.lang.String` on your classpath can never replace the real one — a core security guarantee.

Three phases in detail:

Loading: Reads the `.class` file and creates a `java.lang.Class` object in the Heap representing that class.

Linking — three sub-steps:

1. **Verification** — Bytecode verifier checks that bytecode is structurally valid and does not violate access rules. Defense against corrupt or malicious class files.
2. **Preparation** — Static variables are allocated and set to **default values only** (0, null, false). Your programmer-assigned values are NOT applied yet.

3. **Resolution** — Symbolic references (method and field names stored as strings in the constant pool) are replaced with direct memory references.

Initialization: Static initializer blocks and static field assignments (`static int x = 100`) execute top to bottom. Runs exactly once per class. JVM guarantees this is thread-safe.

2. Runtime Data Areas

Heap — Shared by All Threads

All object instances and arrays live here. Divided into generations:

- **Young Generation** — New objects land in *Eden Space*. When Eden fills, Minor GC runs. Survivors copy to a *Survivor space* (S0 or S1, alternating). After surviving enough GC cycles (default threshold: 15), objects promote to Old Gen.
- **Old Generation (Tenured)** — Long-lived objects. When it fills, Major or Full GC runs — far more expensive than Minor GC.
- **Metaspace (Java 8+)** — Lives in **native memory, outside the JVM heap**. Replaced PermGen. Stores class metadata, method bytecode, and constant pools. Grows dynamically — a classloader leak causes it to grow until the OS runs out.

JVM Stack — One Per Thread

Each thread has its own stack. It holds **Stack Frames** — one per active method call. Frame is pushed on method entry, popped on return.

Each frame contains:

- *Local Variable Array* — Parameters and local variables. Slot 0 in instance methods is always `this`.
- *Operand Stack* — Scratch space where bytecode instructions push operands and pop results.
- *Constant Pool Reference* — Points back to the class runtime constant pool.

`StackOverflowError` = no space for a new frame. Usually caused by infinite recursion. Stack size: `-Xss` (default 512KB–1MB per thread).

PC Register — One Per Thread

Holds the address of the currently executing JVM instruction for that thread. Each thread has its own PC so all threads execute independently.

Native Method Stack — One Per Thread

Like the JVM Stack but for native (C/C++) method calls via JNI. When `System.currentTimeMillis()` runs, its C call frame lives here.

Method Area

A logical memory region (part of Metaspace in modern JVMs) shared by all threads. Stores per-class data: the runtime constant pool, field and method signatures, and method bytecode.

3. Execution Engine

The Execution Engine reads and runs bytecode using two complementary mechanisms.

Interpreter: Reads and executes bytecode one instruction at a time. Starts immediately but is slow for frequently-called code — the same decoding overhead repeats on every call.

JIT Compiler (Just-In-Time): Detects hot methods (called frequently) and compiles them to native machine code. That native code is cached and runs directly on the CPU — no bytecode decoding overhead at all.

HotSpot uses **Tiered Compilation** with two JIT compilers:

Tier	Compiler	What it does
0	Interpreter	All code starts here
1–2	C1 — fast	Quick compile, basic optimization, adds profiling probes
3–4	C2 — optimizing	Deep optimization using C1 profiling data

C1 activates at around 200 invocations. C2 activates at around 10,000 invocations. Tier 4 output rivals hand-written C++ in throughput.

Key HotSpot optimizations:

- **Method Inlining** — JIT copies the called method body directly into the caller. No frame push or pop. Getters and small helpers are almost always inlined.
- **Escape Analysis** — If an object never leaves a method (not stored in a field, not returned, not passed to another thread), JIT allocates it on the stack instead of the heap — zero GC pressure.

- **On-Stack Replacement (OSR)** — Replaces an interpreted hot loop with compiled native code mid-execution, without waiting for the next call.

4. Native Method Interface (JNI)

JNI is the bridge between Java and native C/C++ code. Some operations require OS-level access that pure Java cannot provide.

```
public class NativeExample {
    public native int add(int a, int b); // body implemented in C
    static {
        System.loadLibrary("myNativeLib"); // loads .dll or .so
    }
}
```

Commonly used JDK methods that are native under the hood: `System.currentTimeMillis()`, `System.arraycopy()`, `Object.hashCode()`, and all file I/O and networking ultimately calls OS system calls through JNI.

1.3 Unicode vs ASCII

To understand why this matters in Java specifically, you first need to understand the problem that Unicode was built to solve.

ASCII — American Standard Code for Information Interchange

ASCII was created in the early 1960s for American English. It maps 128 characters to the numbers 0–127, using only 7 bits per character.

Range	Content
0–31	Control characters: null (0), tab (9), newline (10), carriage return (13)
32	Space
48–57	Digits 0–9
65–90	Uppercase A–Z
97–122	Lowercase a–z

Range	Content
33–47, 58–64	Symbols: ! " # $ % & ' ( ) * + , - . / : ; < = > ? @

ASCII works perfectly for English text. It has zero support for accented characters (é, ñ, ü), non-Latin scripts (Arabic, Chinese, Hindi, Japanese, Korean), currency symbols (€, ₹, ¥), or mathematical notation.

Extended ASCII: Various vendors extended ASCII to 8 bits (256 characters) to add European characters. ISO-8859-1 (Latin-1) is one example — it covers Western European languages in the 128–255 range. The problem: every country and vendor created **incompatible** extensions. A byte value of 200 means È in ISO-8859-1 but something entirely different in IBM EBCDIC or Windows-1252. And none of these 8-bit extensions could represent Chinese, Japanese, Arabic, or Hindi at all — those scripts alone require thousands of characters.

Unicode — The Universal Standard

Unicode was created in 1991 by the Unicode Consortium to be a **single encoding standard** for every character in every human language — plus technical symbols, mathematical notation, emoji, and historic scripts.

Code Points: Unicode assigns every character a unique number called a **code point**, written as `U+XXXX` (hexadecimal). Unicode can represent 1,114,112 code points (`U+0000` to `U+10FFFF`).

```
U+0041  →  A       (Latin capital letter A)
U+0061  →  a       (Latin small letter a)
U+00E9  →  é       (Latin e with acute accent)
U+0928  →  न       (Devanagari letter NA — used in Hindi)
U+4E2D  →  中       (Chinese character for middle / China)
U+1F600 →  😀      (Grinning Face emoji)
U+20B9  →  ₹       (Indian Rupee sign)
```

The Unicode code space is divided into 17 **planes**. Plane 0 (`U+0000` to `U+FFFF`) is the **Basic Multilingual Plane (BMP)** and contains almost all commonly used characters from all modern scripts. Planes 1–16 (called supplementary planes) contain less common characters, historic scripts, and emoji.

Unicode Encoding Schemes — UTF-8, UTF-16, UTF-32

Unicode defines **what** number a character gets (its code point). The encoding scheme defines **how** that number is stored as bytes. There are three main encodings:

UTF-32 (32-bit) Stores every code point in exactly 4 bytes. Simple — `U+004E` is stored as `0x0000004E`. But wasteful: English text uses 4 bytes per character when 1 would be enough. Rarely used except for internal processing.

UTF-16 (16-bit) Stores BMP characters (`U+0000` to `U+FFFF`) in exactly **2 bytes**. Characters in supplementary planes (`U+10000` to `U+10FFFF`) require 4 bytes using **surrogate pairs** — a pair of 16-bit values (high surrogate `U+D800` – `U+DBFF` + low surrogate `U+DC00` – `U+DFFF`) that together encode the supplementary character.

```
BMP character  U+004E (N)  →  0x004E           (2 bytes)
Emoji 😀       U+1F600     →  0xD83D 0xDE00    (4 bytes, surrogate pair)
```

UTF-8 (8-bit, variable-length) The most widely used encoding today. ASCII characters (`U+0000` – `U+007F`) are stored in exactly **1 byte** — identical to ASCII. Other characters use 2, 3, or 4 bytes:

Code point range	Bytes used	Characters covered
U+0000 – U+007F	1 byte	ASCII (English, digits, punctuation)
U+0080 – U+07FF	2 bytes	Latin extended, Greek, Cyrillic, Hebrew, Arabic
U+0800 – U+FFFF	3 bytes	Most CJK characters, other BMP scripts
U+10000 – U+10FFFF	4 bytes	Emoji, supplementary scripts, historic languages

UTF-8 is the standard on the web, in Linux/macOS filesystems, in JSON, and in Java source files. It is backward compatible with ASCII — any ASCII file is also a valid UTF-8 file.

Why Java Uses UTF-16 Internally

Java was designed in 1991–1995. At that time, Unicode promised it would fit all the world's characters into 16 bits (65,536 code points). Java's `char` type was designed as a **16-bit unsigned integer** to hold any Unicode character.

By 1996, Unicode expanded beyond 16 bits. Java was already too widely deployed to change the `char` size. So the decision became permanent: **Java's internal string representation uses UTF-16**.

```
char c = 'A';           // U+0041 — fits in one char (BMP character)
char c = '\u0041';      // same thing using Unicode escape
char c = '\u20B9';      // ₹ Indian Rupee — fits in one char (U+20B9 is BMP)

// Emoji U+1F600 is in the supplementary plane — does NOT fit in one char
String emoji = "\uD83D\uDE00";  // explicit surrogate pair for 😀
String emoji = "😀";             // same — Java stores it as two chars
internally
```

Practical Implications in Java

`String.length()` counts UTF-16 code units, not characters:

```
String emoji = "😀";
System.out.println(emoji.length());             // 2 — two UTF-16 code units
System.out.println(emoji.codePointCount(0, 2)); // 1 — one actual Unicode
character

String text = "Hello";
System.out.println(text.length());              // 5 — correct for BMP-only
strings
```

`charAt()` can return half a surrogate pair:

```
String emoji = "😀";
System.out.println(emoji.charAt(0));       // garbage — first half of
surrogate pair
System.out.println(emoji.codePointAt(0)); // 128512 — correct code point
(U+1F600)
```

Safe iteration over all Unicode characters:

```
// WRONG for strings containing emoji or supplementary characters:
for (int i = 0; i < str.length(); i++) {
    char c = str.charAt(i); // may split a surrogate pair
}

// CORRECT — iterate over code points:
str.codePoints().forEach(cp -> {
    String ch = Character.toString(cp); // works for all Unicode characters
    System.out.println(ch);
});
```

Unicode escape in Java source code — important trap:

Java processes `\uXXXX` escapes **before parsing** the source. This means they can appear anywhere — inside string literals, comments, even in operator positions:

```
String s = "\u0048\u0065\u006C\u006C\u006F"; // "Hello"

// DANGEROUS: \u000A is the newline character
// If it appears in a comment, it creates a real newline during compilation:
// \u000A System.out.println("this code actually runs!");
```

Java 18+ — UTF-8 as the default charset:

Before Java 18, `Charset.defaultCharset()` returned the OS default — UTF-8 on Linux, Windows-1252 on Windows. A text file written on Windows would appear garbled on Linux. This was a notorious source of hard-to-reproduce bugs.

Since Java 18, the default charset is **always UTF-8** regardless of OS or locale. Java programs now behave consistently across all platforms for text I/O.

```
// Before Java 18 — platform-dependent, could vary:
Charset.defaultCharset(); // UTF-8 on Linux, Windows-1252 on Windows

// Java 18+ — always UTF-8:
Charset.defaultCharset(); // UTF-8 on every OS

// Best practice regardless of version — always specify charset explicitly:
Files.readString(path, StandardCharsets.UTF_8);
new InputStreamReader(stream, StandardCharsets.UTF_8);
```

Summary — ASCII vs Unicode vs Java char

	ASCII	Unicode	Java `char`
Size	7 bits (128 chars)	1,114,112 code points	16 bits (2 bytes)
Coverage	English only	All human languages	BMP only (U+0000–U+FFFF)
Encoding	Fixed 1 byte	UTF-8, UTF-16, UTF-32	UTF-16 internally

	ASCII	Unicode	Java `char`
Supplementary chars	No	Yes (U+10000+)	Needs 2 chars (surrogate pair)

Review Questions — Basic and Senior Level

Attempt every question first without looking at the answers. All answers and explanations are consolidated in the next section.

Part A — Multiple Choice (Fundamentals)

Q1. Which component contains the `javac` compiler?

- (a) JVM
- (b) JRE
- (c) JDK
- (d) JIT

Q2. What does `javac` produce from a `.java` file?

- (a) Native machine code for the current OS
- (b) Bytecode in a `.class` file
- (c) Assembly code
- (d) A `.jar` file

Q3. Which is the correct classic entry-point signature?

- (a) `public static int main(String[] args)`
- (b) `public static void main(String[] args)`
- (c) `public static main(String[] args)`
- (d) `private static void main(String[] args)`

Q4. Which ClassLoader loads `java.lang.String`?

- (a) Application ClassLoader
- (b) Platform ClassLoader
- (c) Bootstrap ClassLoader
- (d) A custom ClassLoader

Q5. For `static int x = 10;`, what is the value of `x` at the end of the **Preparation** phase?

- (a) 10
- (b) 0
- (c) null
- (d) Undefined

Q6. Which of the following is NOT per-thread?

- (a) PC Register
- (b) JVM Stack
- (c) Native Method Stack
- (d) Heap

Q7. Where does Metaspace live?

- (a) Inside the Young Generation
- (b) Inside the Old Generation
- (c) In native (off-heap) memory
- (d) On the thread stack

Q8. What most commonly causes `StackOverflowError`?

- (a) Creating too many objects
- (b) Infinite or very deep recursion
- (c) Too many loaded classes
- (d) Running out of disk space

Q9. Which comment style is used by the `javadoc` tool?

- (a) `// text`
- (b) `/* text */`
- (c) `/** text */`
- (d) `# text`

Q10. How many `public` top-level classes can one `.java` file contain?

- (a) Any number
- (b) Exactly one
- (c) At most one

- (d) At most two

Q11. Which statement about Java is TRUE?

- (a) Java is purely object-oriented
- (b) Java supports pointer arithmetic
- (c) Java is statically typed
- (d) Java bytecode is specific to one OS

Q12. Which edition adds Servlets, JPA and JAX-RS on top of the core platform?

- (a) Java SE
- (b) Java ME
- (c) Jakarta EE
- (d) JavaFX

Q13. Which of these are LTS releases?

- (a) 9, 10, 12
- (b) 8, 11, 17, 21
- (c) 14, 15, 16
- (d) 13, 18, 19

Q14. What does the JIT compiler do?

- (a) Converts `.java` to `.class`
- (b) Compiles hot bytecode to native machine code at runtime
- (c) Collects garbage
- (d) Verifies bytecode

Q15. How many bits does classic ASCII use per character?

- (a) 7
- (b) 8
- (c) 16
- (d) 32

Q16. How many bytes does UTF-8 use for the letter `A`?

- (a) 1
- (b) 2
- (c) 3

- (d) 4

Q17. What does `"😀".length()` return?

- (a) 1
- (b) 2
- (c) 4
- (d) 0

Q18. What is Java's default charset since Java 18?

- (a) The OS default
- (b) ISO-8859-1
- (c) UTF-8
- (d) UTF-16

Q19. Which JVM memory area stores class metadata in Java 8 and later?

- (a) PermGen
- (b) Eden
- (c) Metaspace
- (d) Survivor space

Q20. Which phase assigns the programmer-written values to static fields?

- (a) Loading
- (b) Verification
- (c) Preparation
- (d) Initialization

Part B — Short Answer (Fundamentals)

Q21. Explain Write Once, Run Anywhere. What exactly is platform-independent, and what is platform-specific?

Q22. Differentiate JDK, JRE and JVM. Which one do you install on a build server, and which on a production server?

Q23. Walk through what happens, step by step, from typing `javac Hello.java` to seeing output from `java Hello`.

Q24. Describe the three phases of class loading and name the sub-steps of Linking.

Q25. What is stored in a stack frame? What happens to it when a method returns?

Q26. List the 8 features of Java you consider most important and give one line on each.

Q27. Why does `"Hello".length()` give the right answer but `"😀".length()` does not give 1?

Q28. Why was PermGen replaced by Metaspace?

Part C — Senior Level

S1. Explain the parent delegation model. Give two real situations in which frameworks intentionally break it, and the risk of doing so.

S2. What is the difference between `Class.forName("X")` and `ClassLoader.loadClass("X")`? When exactly is a class initialized?

S3. A Tomcat app is hot-redeployed repeatedly and eventually fails with `OutOfMemoryError: Metaspace`. Explain the root cause and how you would diagnose and fix it.

S4. Describe tiered compilation. What is deoptimization, and why can a JIT-compiled method go back to the interpreter?

S5. "All objects are allocated on the heap." True or false? Justify with escape analysis and scalar replacement.

S6. Your microservice starts slowly and is slow for the first minute in a container, then becomes fast. Explain why, and list at least three strategies to improve startup and warm-up.

S7. Compare running on HotSpot with a GraalVM native image. What do you gain, and what do you give up?

S8. Java strings are described as UTF-16, but since Java 9 they use compact strings. Explain what that means and how it affects memory.

S9. You must reverse a user-supplied string that may contain emoji. Why does a naive `charAt` loop corrupt it, and what are the safe options?

S10. What did JEP 400 (Java 18) change? What bugs did it fix, and what should you still do explicitly in your code?

S11. Where does the String pool live, and how has that changed across versions? How is the constant pool different from the String pool?

S12. The JVM is a specification, and Kotlin, Scala and Groovy all run on it. What makes that possible, and what role does `invokedynamic` play?

S13. Differentiate `OutOfMemoryError: Java heap space`, `Metaspace`, `unable to create native thread`, and `StackOverflowError`. For each, name the memory area involved and the first diagnostic step.

Answers — Basic and Senior Level

Part A — Multiple Choice

Q	Answer	Explanation
1	(c) JDK	The JDK bundles the compiler and dev tools on top of the JRE.
2	(b)	`javac` emits platform-independent bytecode. The JVM translates it to native code at runtime.
3	(b)	The classic launcher looks for exactly `public static void main(String[])`. Note: Java 25 finalized simplified launch forms (instance main, compact source files), but the classic signature remains the canonical form.
4	(c)	Core classes are always loaded by Bootstrap. Parent delegation prevents anything else from replacing them.
5	(b) 0	Preparation assigns default values only. `10` is assigned in Initialization.
6	(d) Heap	The Heap is shared by all threads. The other three are private to each thread.
7	(c)	Metaspace uses native memory, outside the JVM heap.
8	(b)	Each call pushes a frame. Unbounded recursion exhausts the stack.
9	(c)	`/** ... */` is the Javadoc form.

Q	Answer	Explanation
10	(c)	At most one public top-level class, and its name must match the file name. A file may have zero public classes.
11	(c)	Java is statically typed. It is not purely OO (primitives), has no pointer arithmetic, and bytecode is OS-independent.
12	(c)	Jakarta EE (formerly Java EE) layers enterprise APIs over SE.
13	(b)	LTS releases: 8, 11, 17, 21, and 25.
14	(b)	JIT compiles hot bytecode to native code while the program runs.
15	(a)	ASCII is 7-bit, 128 characters.
16	(a)	UTF-8 is backward compatible with ASCII, so ASCII characters use 1 byte.
17	(b) 2	The emoji is a surrogate pair, which is two UTF-16 code units. Use `codePointCount` for 1.
18	(c)	Since Java 18 the default charset is always UTF-8.
19	(c)	Metaspace replaced PermGen in Java 8.
20	(d)	Static initializers and assignments run in Initialization.

Part B — Short Answer (model answers)

Q21 — WORA. The `.class` bytecode is platform-independent: the same file is valid everywhere. The JVM is platform-specific: each OS and CPU has its own JVM that turns bytecode into native instructions. Java also fixes primitive sizes (an `int` is always 32 bits), so behavior does not vary by platform.

Q22 — JDK vs JRE vs JVM. JVM executes bytecode. JRE is the JVM plus the standard class libraries, which is what you need to run programs. JDK is the JRE plus the compiler and developer tools, which is what you need to build programs. A build server needs the JDK. Production needs only a runtime, but since Java 11 there is no standalone JRE download, so you either install a JDK or ship a trimmed runtime built with `jlink`.

Q23 — End to end. (1) `javac` lexes, parses, type-checks and writes `Hello.class`. (2) `java Hello` starts the JVM, which initializes memory areas. (3) The ClassLoader locates and loads `Hello.class` (after delegating to its parents). (4) Linking: bytecode verification, static default allocation, reference resolution. (5) Initialization runs static blocks. (6) The JVM invokes `main` on the main thread: the interpreter executes it, and hot code gets JIT-compiled. (7) Output goes through `System.out`, which reaches the OS via native calls. (8) When `main` and all non-daemon threads finish, the JVM exits.

Q24 — Class loading phases. Loading (find the bytes, create the `Class` object), Linking, and Initialization. Linking has three sub-steps: Verification (bytecode is valid and safe), Preparation (allocate statics with default values), Resolution (symbolic references become direct references). Initialization then runs static initializers once, thread-safely.

Q25 — Stack frame. A frame holds the local variable array (parameters and locals, with `this` in slot 0 for instance methods), the operand stack for intermediate computation, and a reference to the class's runtime constant pool. On return, the frame is popped and its locals vanish. Objects they referenced survive on the heap if still reachable.

Q26 — Features. Platform independent (bytecode + JVM), object-oriented (encapsulation, inheritance, polymorphism, abstraction), robust (strong typing, exceptions, GC), secure (verifier, sandbox, no pointer arithmetic), multithreaded (built-in concurrency APIs), architecture-neutral (fixed primitive sizes), high performance (JIT), dynamic and distributed (runtime class loading and reflection, networking APIs).

Q27 — Length of emoji. `length()` counts UTF-16 code units, not characters. `Hello` is five BMP characters, so five units. An emoji lies outside the BMP and takes a surrogate pair of two units, so `length()` returns 2. Use `codePointCount(0, s.length())` to count actual characters.

Q28 — PermGen to Metaspace. PermGen had a fixed maximum size, so large apps or repeated redeploys hit `OutOfMemoryError: PermGen space`, and tuning it was guesswork. Metaspace uses native memory and grows on demand, with an optional cap via `-XX:MaxMetaspaceSize`. It also simplified GC of class metadata and was a step toward unifying the HotSpot and JRockit designs.

Part C — Senior Level (model answers)

S1 — Parent delegation. A loader asks its parent first and loads the class itself only if the parent fails. This gives consistency (one `String` for everyone) and security (core classes cannot be spoofed). It is deliberately broken when isolation or versioning is needed. Tomcat gives each web app its own loader that prefers its own `WEB-INF/lib` classes, so

two apps can use different versions of the same library. OSGi and plugin systems use per-module loaders with a graph of imports. JDBC and JNDI/SPI use the thread context ClassLoader so core code can load implementations supplied by the application. Risks include `ClassCastException` between "identical" classes loaded by different loaders (a class is identified by name plus loader), `LinkageError`, and classloader leaks.

S2 — `forName` vs `loadClass`. `Class.forName(name)` loads, links and initializes the class, so static blocks run. `ClassLoader.loadClass(name)` only loads, with no initialization until first active use. That is why JDBC drivers historically used `Class.forName`: the static block registered the driver. A class is initialized on first active use: `new`, calling a static method, reading or writing a non-constant static field, `Class.forName`, initialization of a subclass (which first initializes the superclass), and being the main class. Reading a compile-time constant (`static final int X = 5`) does not trigger initialization, because the value is inlined by the compiler.

S3 — Metaspace leak on redeploy. Class metadata can be unloaded only when its ClassLoader becomes unreachable. After a redeploy the old webapp loader should be collectable, but anything in a longer-lived loader that still references the old loader or its classes pins all of its metadata. Typical pinning culprits: `ThreadLocal` values left in container threads, JDBC drivers registered with `DriverManager`, non-daemon threads started by the app, static caches in shared libraries, and shutdown hooks. Diagnose with `-XX:+HeapDumpOnOutOfMemoryError` or `jcmd <pid> GC.class_stats` and `jmap -clstats`, then open a heap dump in Eclipse MAT and follow paths to GC roots for the old `WebappClassLoader`. Fix by cleaning `ThreadLocal`s, deregistering drivers, stopping threads in a context-listener shutdown, and avoiding statics in shared libs. A `MaxMetaspaceSize` cap makes the failure fast and visible rather than consuming the host.

S4 — Tiered compilation and deoptimization. Code starts in the interpreter (tier 0). Methods that cross invocation or loop thresholds are compiled by C1 with profiling (tiers 1 to 3), and truly hot ones are recompiled by C2 with aggressive optimization based on collected profiles (tier 4). C2 speculates: for example, "this call site only ever sees class A" so it inlines A's method behind a cheap type guard. If an assumption breaks (a new subclass is loaded, a branch never taken before is taken, a type guard fails), the code hits an uncommon trap and **deoptimizes**: execution state is reconstructed and the method returns to the interpreter, to be re-profiled and recompiled. Deopts are normal, but repeated deopt storms show up as performance cliffs.

S5 — Are objects always on the heap? Semantically yes, objects are heap objects. Physically, no. After inlining, the JIT's escape analysis can prove an object never escapes the method or thread. It can then apply scalar replacement, breaking the object into fields

held in registers or stack slots, so no allocation happens at all. Related optimizations: lock elision for non-escaping objects, and removal of dead allocations. HotSpot does not do true stack allocation of whole objects; scalar replacement is the practical mechanism. Escape analysis only runs on JIT-compiled code, and only when inlining succeeds, so it is not a guarantee to rely on.

S6 — Warm-up. The JVM must load and verify classes, run in the interpreter, and profile before C2 produces peak code. In containers, small CPU quotas make the JIT compiler threads compete with application threads, so warm-up stretches further. Strategies: Class Data Sharing (CDS and AppCDS) to cut class load and verify time; trimming dependencies and component scanning (for Spring, lazy init, indexed scanning); tuning `-XX:TieredStopAtLevel=1` for short-lived tools; giving the container enough CPU during startup; AOT or caching approaches such as Project Leyden's ahead-of-time cache in recent JDKs; checkpoint and restore with CRaC where available; GraalVM native image for near-instant startup; and readiness probes plus synthetic warm-up traffic before joining the load balancer.

S7 — HotSpot vs native image. Native image compiles ahead of time to a standalone executable: very fast startup, low memory footprint, and no JVM install. The costs: a closed-world assumption, so reflection, dynamic proxies, resources and serialization must be declared in configuration; no runtime class loading of unknown classes; peak throughput is often lower than C2 because there is no runtime profiling (profile-guided optimization partly closes the gap); long build times; harder debugging and diagnostics. HotSpot gives the opposite: slower startup, higher memory, but dynamic features and the best long-running throughput.

S8 — Compact strings. Since Java 9, `String` is backed by a `byte[]` plus a coder flag. If every character fits in Latin-1 the string uses one byte per character, otherwise it uses two (UTF-16). The semantics are unchanged (`charAt`, `length` still behave as UTF-16 code units), but heap usage drops significantly for typical English or Western-European text, which often reduces GC pressure. You can disable it with `-XX:-CompactStrings`, which is rarely useful.

S9 — Reversing emoji strings. A `charAt` loop reverses the two halves of each surrogate pair, producing invalid sequences. Safe options: `new StringBuilder(s).reverse()` (documented to treat surrogate pairs as single characters), or iterate with `codePoints()` and rebuild. Note that code points are still not user-perceived characters: combining marks and emoji with ZWJ sequences (family emoji, skin tones) span several code points, so for true grapheme handling use `java.text.BreakIterator.getCharacterInstance()` or ICU4J.

S10 — JEP 400. It made UTF-8 the default charset for the standard Java APIs (`FileReader`, `new String(bytes)`, `Files` methods that default, etc.) regardless of OS or locale, ending the "works on my Linux box, garbled on Windows" class of bugs. It does not change everything: the console and native interfaces can still use platform encodings (see `stdout.encoding` and `native.encoding`), and old files written in a legacy encoding still need that encoding specified. Best practice is unchanged: always pass `StandardCharsets.UTF_8` explicitly when reading or writing text, and declare UTF-8 in HTTP headers, database connections and compiler settings (`-encoding UTF-8`).

S11 — String pool and constant pool. The String pool (intern table) holds canonical `String` instances for literals and `intern()` calls. It lived in PermGen up to Java 6, and was moved to the **heap in Java 7**, so interned strings became collectable and no longer cause PermGen exhaustion. The runtime constant pool is a per-class structure in Metaspace, built from the class file's constant pool, holding literals and symbolic references; string literal entries there resolve to references into the String pool. So: constant pool is per class and holds many kinds of constants and symbols, the String pool is global and holds unique `String` objects.

S12 — Multi-language JVM. The JVM executes bytecode, not Java. Any compiler that emits valid class files can target it. Kotlin, Scala, Groovy and Clojure do exactly that. `invokedynamic` (Java 7) lets the call-site linkage be decided at runtime through bootstrap methods, which was designed for dynamic languages. Java itself now uses it for lambdas (`LambdaMetafactory`), string concatenation (Java 9+), and records and pattern-matching support.

S13 — Error types.

Error	Memory area	First diagnostic step
`OutOfMemoryError: Java heap space`	Heap	Heap dump (`-XX:+HeapDumpOnOutOfMemoryError`), analyze in MAT for dominator and leak suspects.
`OutOfMemoryError: Metaspace`	Native memory (class metadata)	Look for classloader leaks or runaway dynamic class generation (proxies, scripting). `jcmd` class stats.
`OutOfMemoryError: unable to create native thread`	OS and native memory (thread stacks)	Check thread count (thread dump), ulimit, container limits, `-Xss`; consider thread pools or virtual threads.

Error	Memory area	First diagnostic step
`StackOverflowError`	A thread's JVM stack	Read the repeating frames in the stack trace to find the recursion. Fix the logic, and only then consider raising `-Xss`.

End of Section 1. Next up is Section 2 (Basic Programming). Tell me when you are ready, or flag anything in Section 1 you want expanded first.